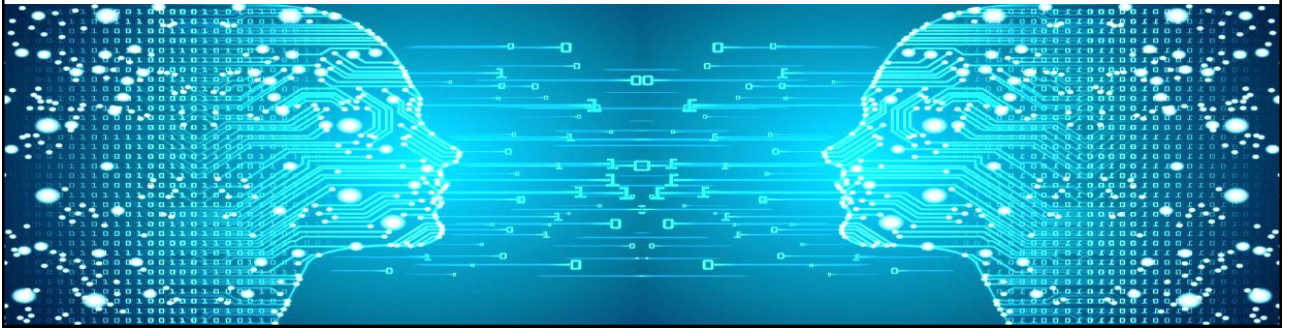
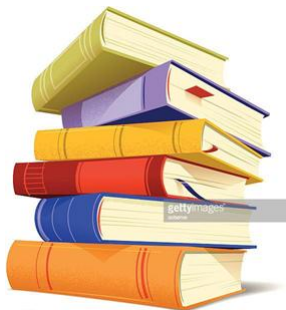


Machine Learning



Neural Networks



- Perceptron Neural Network
- Multilayer Perceptron Networks
- The Backpropagation Algorithm

Perceptron Neural Network



Biological Inspiration

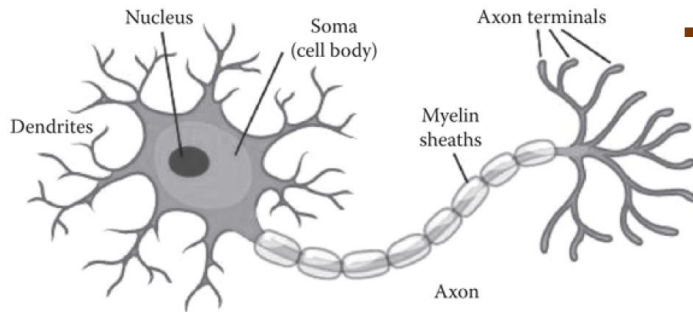
■ What is a Neural Network (NN)?

- A model of reasoning **inspired by biological neural networks** (central nervous system in an animal brain).
- The human brain contains about **10 billion neurons** and **60 trillion connections (synapses)**.

■ Historical Milestone

- The first conceptual model of a neural network was introduced in **1943** by **Warren McCulloch and Walter Pitts**.
- Subsequently, **Artificial Neural Networks (ANNs)** have become one of the most powerful tools for **pattern recognition, regression, and classification**.

Biological Inspiration



■ The Biological Neuron:

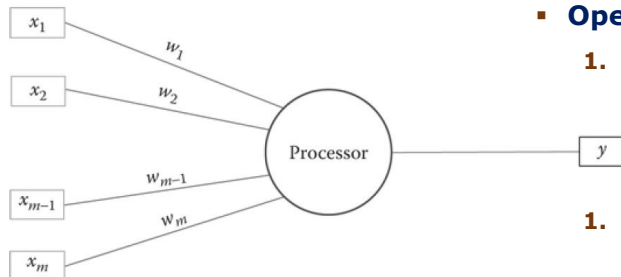
- **Dendrites:** Receive messages/signals from other neurons.
- **Soma (Cell Body):** Signal travels here after reception.
- **Axon:** A single long fiber where the signal travels down to the synapse, moving to the next neuron.

Perceptron Model: Structure and Function

■ The Simplest ANN (artificial neural networks):

- A perceptron consists of a **single neuron**.
- Invented in **1957** by **Frank Rosenblatt**.
- Receives multiple inputs ($a_1, \dots, a_m$ or $x_1, \dots, x_m$) and produces a single output.
- Used to classify **linearly separable classes**.

Perceptron Model: Structure and Function



Operation (Refer to Figure 6.2, p.128)

- 1. Weighted Sum:** Inputs (x_j) are multiplied by corresponding weights (w_j) and summed up:

$$\sum_{j=1}^m w_j \cdot x_j = w_1 \cdot x_1 + \dots + w_m \cdot x_m$$

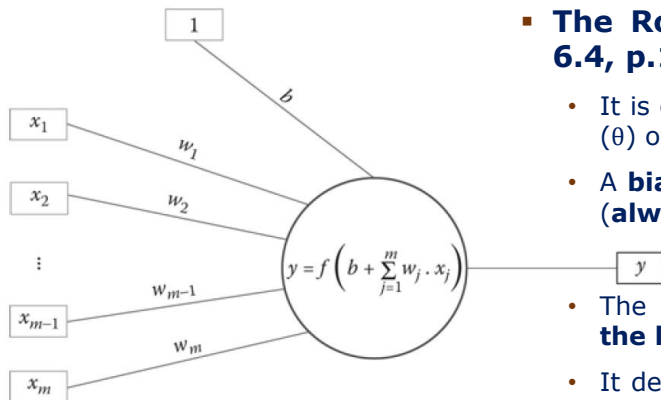
- 1. Activation:** The weighted sum is passed to an **activation function**.
- 2. Output:** The result is compared to a predetermined threshold (θ):
 - If Weighted Sum $> \theta$, the perceptron **outputs 1** (fires).
 - Otherwise, it **outputs 0**.

Common Activation Functions

Common Activation Functions (Refer to Figure 6.3, p.129)

Activation Function	Formular
Step Function	$f(x) = \begin{cases} 0, & x < 0 \\ 1, & x \geq 0 \end{cases}$
Sign Function	$f(x) = \begin{cases} -1, & x < 0 \\ 0, & x = 0 \\ 1, & x > 0 \end{cases}$
Linear function	$f(x) = x$
Sigmoid function	$f(x) = \frac{1}{1 + e^{-x}}$

The Role of Bias



■ The Role of Bias (b) (Refer to Figure 6.4, p.130)

- It is often more convenient to have a threshold (θ) other than zero.
- A **bias (b)** is introduced as an additional input (**always 1**) to the perceptron.
- The bias **moves the threshold function to the left or right.**
- It determines *when* the perceptron fires, along with the weights.

Perceptron Training Algorithm

■ Goal:

- Determine the **optimal weights** (w) and **bias** (b) at which the perceptron fires correctly for the training data.

■ Training Setup

- The training data consists of classified instances (x_j, y_j) , where $y_j \in \{-1, 1\}$ (assuming the Sign activation function).
- The feature vector x_j is extended to include the bias input (which is always 1).

Perceptron Training Algorithm

- **The Perceptron Learning Rule (Epoch)**

1. **Initialize:** Generate random values for the weight vector w .
2. **Calculate Error (E_j):** The error for classifying instance x_j is defined as the difference between the actual class (y_j) and the predicted sign: $E_j = y_j - \text{sign}(w^T \cdot x_j)$
3. **Adjust Weights (Δw):** If the error is non-zero ($E_j = \pm 2$), the weights are corrected using a **learning rate** ($\alpha \in (0,1)$): $\Delta w = \alpha x_j E_j$
4. **Update Weights:** $w_{\text{new}} = w_{\text{old}} + \Delta w$

- **Epoch:** Applying this round of calculation and adjustment to **all instances** in the training set.

Multilayer Perceptron Networks



Multilayer Perceptron Networks (MLP)

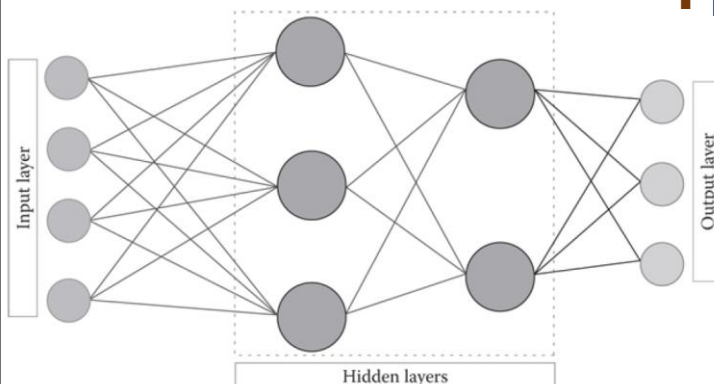
▪ The XOR Problem and Perceptron Limits

- A single perceptron can only solve problems involving **linearly separable classes**.
- The logical **XOR problem** (Exclusive OR, Table 6.1) is the most common simple example of a **nonlinearly separable problem**.
- The true and false instances of XOR cannot be separated by a single straight line (hyperplane) in the feature space (Refer to Figure 6.5).

Multilayer Perceptron Networks (MLP)

▪ MLP Structure

- MLP is used to solve nonlinearly separable problems.
- It consists of **multiple layers of perceptrons** (neurons).
- Decision boundaries in an MLP are geometrically **more complex** than a simple hyperplane.
- Each perceptron receives input from the previous layer.



Multilayer Perceptron Networks (MLP)

▪ MLP Training Overview

- Random weights and biases are initialized.
- An input instance is passed forward layer by layer to compute the predicted output.
- The error at the output layer is calculated (difference between actual and predicted output).
- Weights are then corrected **backward** from the output layer towards the input layer.

The Backpropagation Algorithm



Overview

- **Purpose**

- A learning algorithm used to train MLPs.

- **Two Stages**

- **Feed-forward Stage:** Inputs are fed forward, and the output is computed at the hidden and output layers.
- **Backpropagation Stage:** The prediction error computed at the output layer is **propagated backward** to adjust the weights.

Overview

- **Gradient Descent and Activation Functions**

- Backpropagation uses **deterministic optimization** (Gradient Descent method) to minimize the **squared error sum**.
- Gradient Descent requires computing the partial derivatives of the activation function with respect to the weights.
- Therefore, **hard limit functions** (Step and Sign functions) **cannot be used**.

Overview

▪ Required Properties of Activation Function (f)

- It must have a **first derivative** (f').
- It must be a **nondecreasing function** ($f'(x) > 0$).
- It must have horizontal asymptotes at both **0 and 1** (for S_1).
- Both f and f' must be **computable functions**.

Sigmoid Activation Functions

▪ Popular Choice

- The **Sigmoid functions** are the most popular choice for activation functions in Backpropagation because they guarantee the continuity and differentiability of the error function.

▪ Common Sigmoid Functions (Refer to Figure 6.7)

▪ Standard Sigmoid ($S_1(x)$): $S_1(x) = \frac{1}{1+e^{-x}}$

- **Derivative:** $\frac{dS_1(x)}{dx} = S_1(x)(1 - S_1(x))$

▪ Hyperbolic Tangent ($S_2(x)$ or $\tanh$): $S_2(x) = \frac{1-e^{-x}}{1+e^{-x}}$

- **Derivative:** $\frac{dS_2(x)}{dx} = 1 - S_2^2(x)$

Weights Updates in Neural Networks

▪ Error Function (E):

- The total error over the training set (N instances) is typically defined as the Mean Squared Error (MSE): $E = \frac{1}{2} \sum_{j=1}^N (y_j - p_j)^2$ Where y_j is the target output and p_j is the predicted output.

▪ Gradient Descent Rule

- The update in the weight vector (Δw) is proportional to the negative of the gradient of the error function: $\Delta w_j = -\alpha \frac{\partial E}{\partial w_j}$, where α is the learning rate.

Weights Updates in Neural Networks

▪ Weight Correction at Unit i (using S_1):

- The correction applied to weight w_{ij} (connecting input z_j to unit i with target u_i and output o_i) is given by:

$$\Delta w_{ij} = \alpha(u_i - o_i)o_i(1 - o_i)z_j$$

▪ Adjustment Flow:

- Weights are adjusted starting from the **output layer** (weights from hidden layer to output layer) and then sequentially backward to the **hidden layer** (input layer to hidden layer).

▪ Read more:

- <https://machinelearningcoban.com/2017/01/12/gradientdescent/>

I. Perceptron (Foundation)

- Inspired by biological neurons.
- The simplest form of Artificial Neural Network (ANN), consisting of only a single neuron.
- Used for **linearly separable** classification problems.
- The training process involves adjusting **weights** (w) and **bias** (b) based on the **Perceptron Learning Rule**.
- The output uses a **hard threshold function**, such as the **Sign function**.

Multilayer Perceptron (MLP) and Backpropagation

- MLP is used to solve **nonlinearly separable** problems, with the classic example being the **XOR problem**.
- An MLP consists of multiple layers of perceptrons.
- **Backpropagation** is the core learning algorithm for MLPs.
- It operates in two phases: **Feed-forward** and **Backpropagation** (error adjustment).
- It employs the **Gradient Descent** method to minimize the **sum of squared errors**.
- Requires **differentiable activation functions** (e.g., **Sigmoid** $S_1(x)$ or $S_2(x)/\tanh$), since **hard threshold functions** (step/sign) cannot be used.

